Reason. Plan. Act. Physical AI Workshop

Lecture 3b: Day 3 Recap

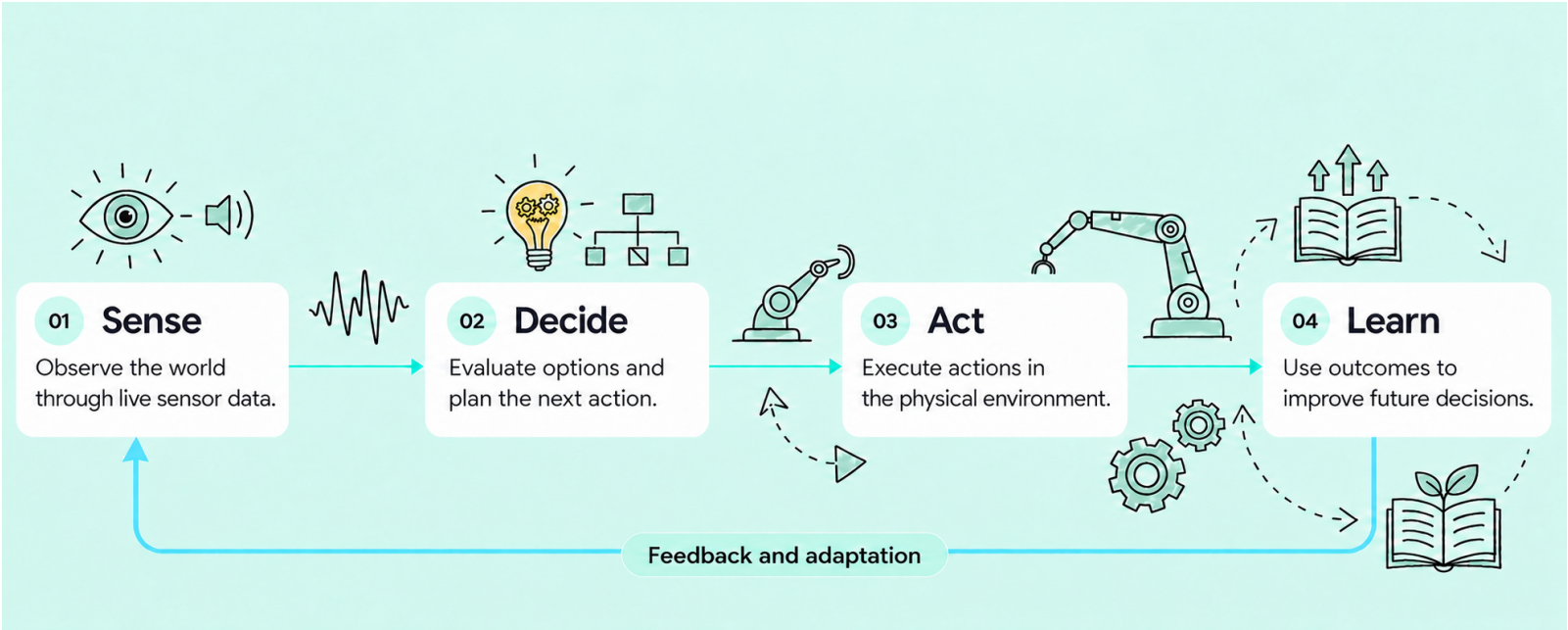
Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology

Recap

The Continuous Closed Loop

Continuous interaction between intelligence and the physical world.

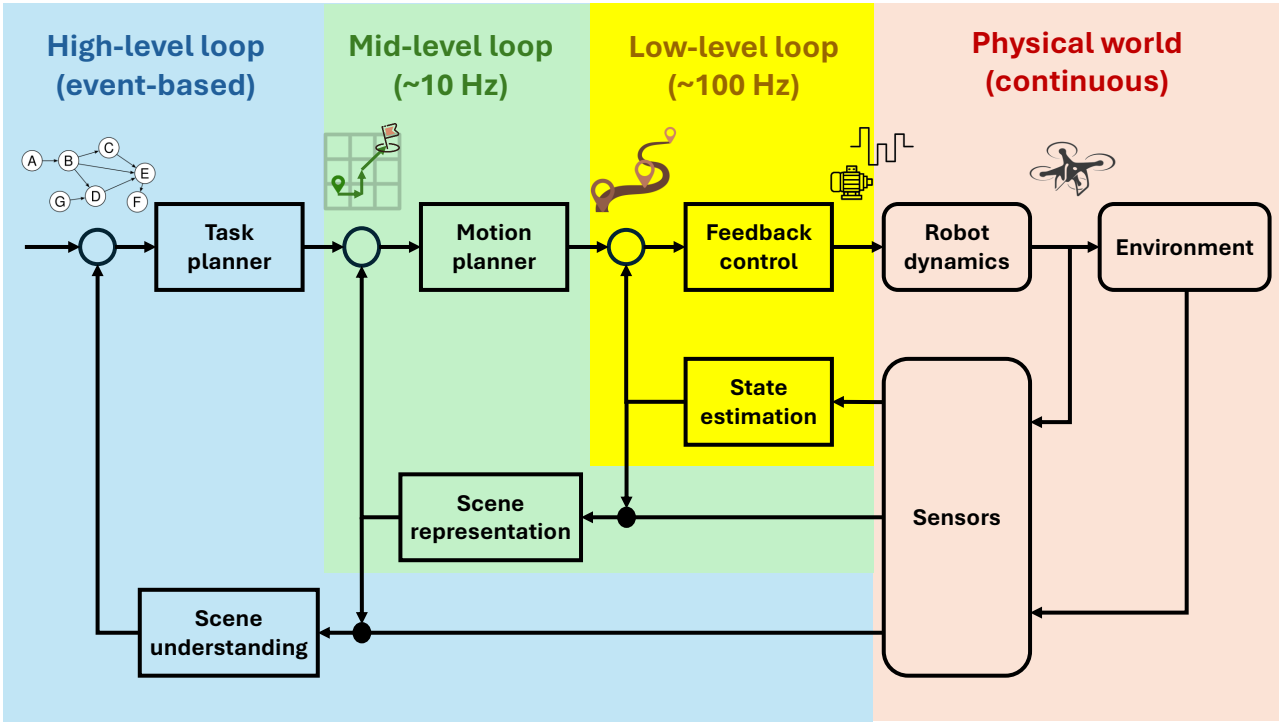


’Pick up the tennis ball’

’Pick up an apple’

’I am hungry’

Layered Decision-Making

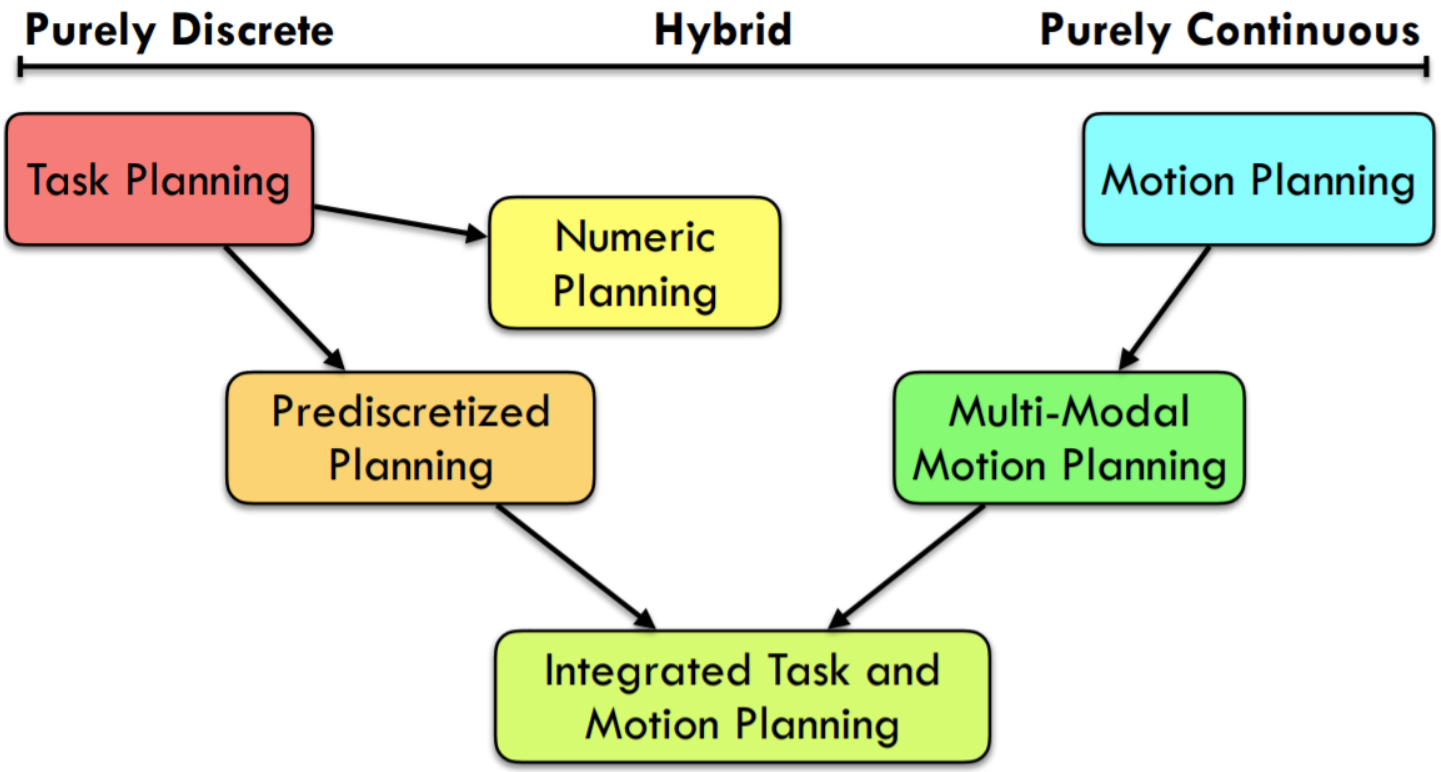


Algorithmic Paradigms for TMP

Three main paradigms:

1. Sequence then Satisfy
2. Satisfy then Sequence
3. Interleaved Task and Motion Planning

Different points in the design space trade off *feasibility guidance* vs. *search flexibility*.


Purely Discrete
Hybrid
Purely Continuous
Task Planning
Numeric Planning
Motion Planning
Prediscretized Planning
Multi-Modal Motion Planning
Integrated Task and Motion Planning

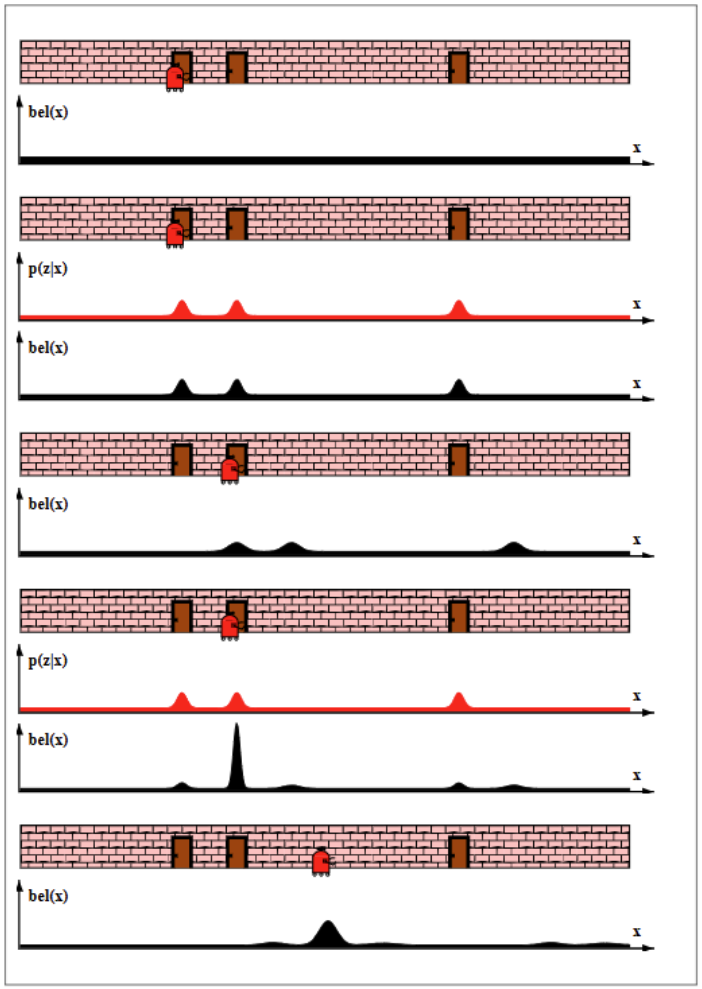
Uncertainty in Robotics

Sources of uncertainty:

- **Perception** – noisy, partial sensor readings (*partial observability*)
- **Action** – actuators slip, grasps fail (*stochastic outcomes*)
- **Model** – dynamics and environment only approximately known

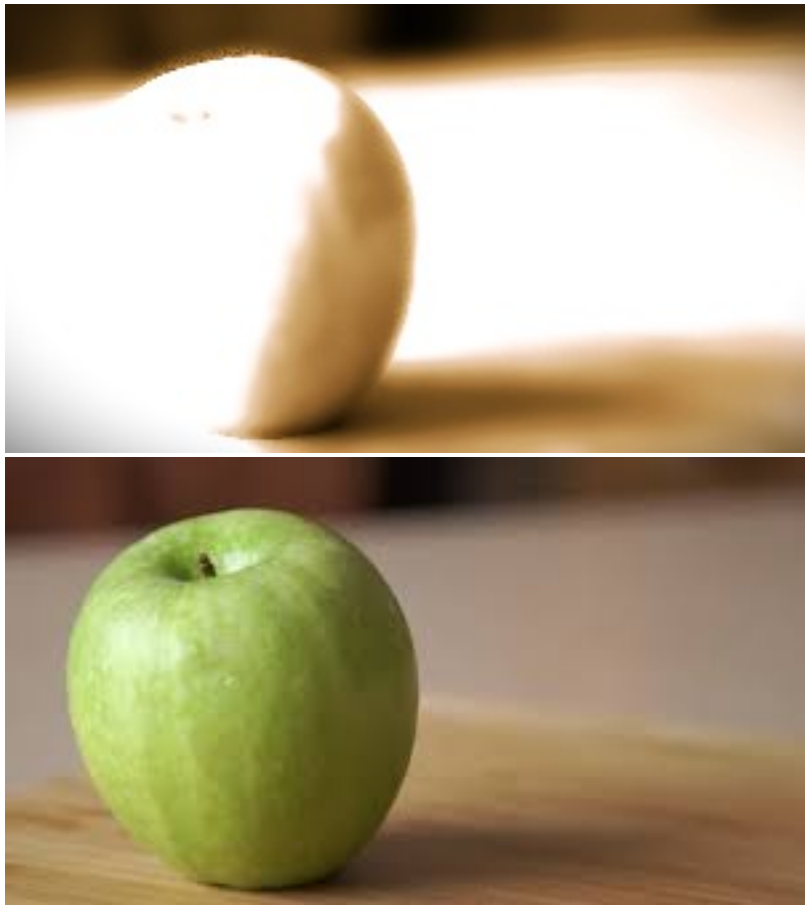
Example - Markov Localization

The agent is given a map of an environment with 3 indistinguishable doors.
Initially, it does not know where it is and needs to localize itself on the map using its sensor data.



From Probabilistic Robotics S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics. Cambridge, MA: MIT Press, 2005.

Partial Observability



Partial Observability Over Time

$t = 3$

$t = 15$

Bayes Filter

Algorithm `Bayes_filter`$(\beta(x_{t-1}), u_t, z_t)$

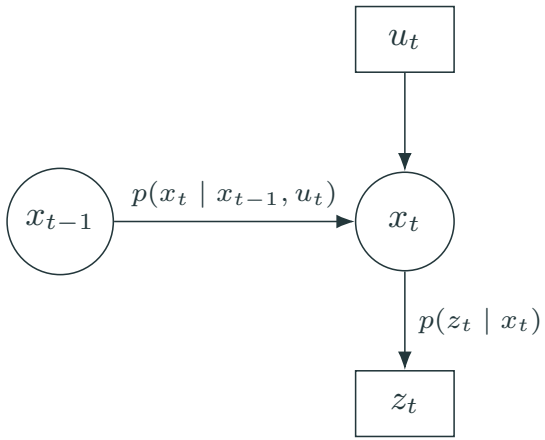
forall x_t do

$$\overline{\beta}(x_t) = \int p(x_t \mid u_t, x_{t-1})\, \beta(x_{t-1})\, dx$$

$$\beta(x_t) = \eta\, p(z_t \mid x_t)\, \overline{\beta}(x_t)$$

end

return $\beta(x_t)$



Formal Definition of TMP #3

A **task and motion planning domain** is defined as $\langle \Sigma, \mathcal{C}, \varphi, \xi, q_0 \rangle$ where

- $\Sigma = \langle S, s_o, S_G, A, \mathcal{P}, \mathcal{C} \rangle$ is a task domain,
- $\mathcal{C}$ is a world configuration space,
- $\varphi : \mathcal{C} \to S$ is a **state estimator function** that maps configurations to task domain states,
- $\xi : A \times \mathcal{C} \to \mathcal{C}^+$ is an **action mapper function** that maps actions into motion plans ($\mathcal{C}^+$ is the set of finite configuration sequences), and
- q_0 is an initial world configuration.

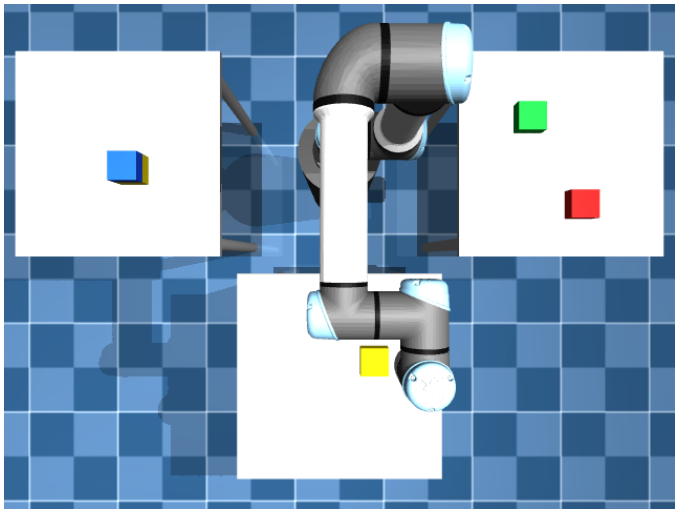
A TMP domain is a task domain and a motion domain with an initial configuration, linked via the state estimator (from motion to task) and action mapper (from task to motion).

Formal Definition of TMP #3

Denote by $\xi(a, q)_{-1}$ the final configuration in motion plan generated by action planner ξ given action aand state s.

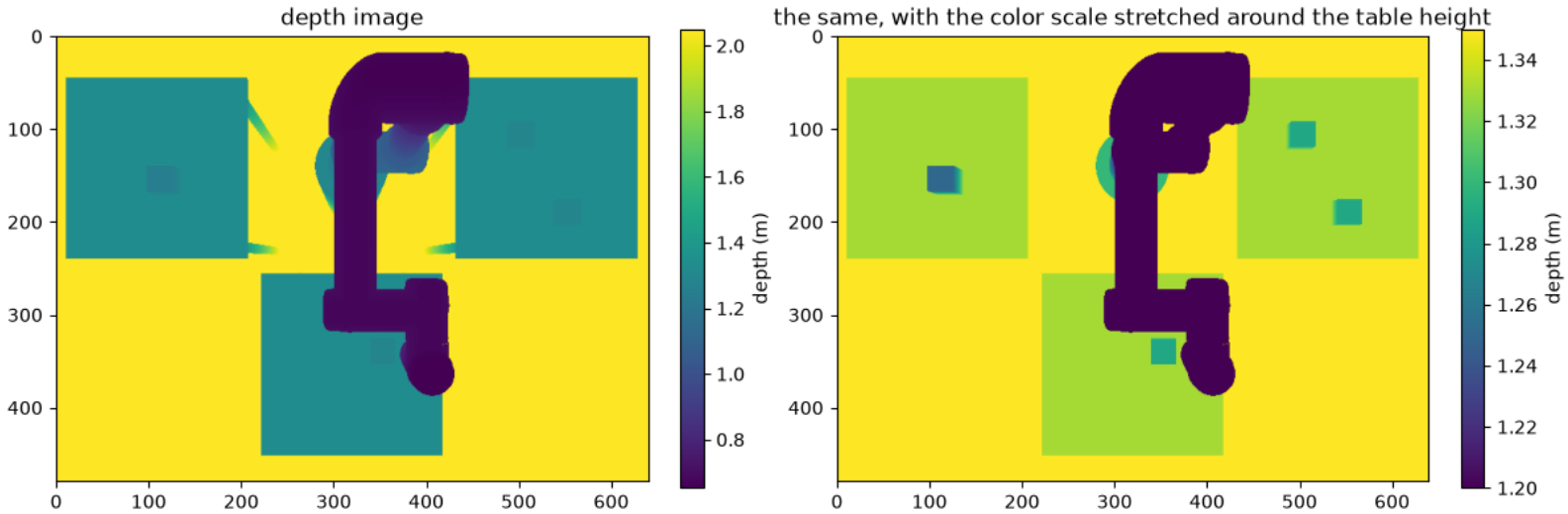
A **task and motion plan** is a task plan $(1, ..., n)$ such that a_1 is applicable from $\varphi(q_0)$, a_2 is applicable from $\varphi(\xi(1, q_0)_{-1})$, a_3 is applicable from $\varphi(\xi(a_1, \xi(a_1, q_0)_{-1}))$, etc., and the final configuration achieved in this manner q_g is mapped to a goal state, i.e., $\varphi(q_g) \in S_G$.

A camera on the ceiling



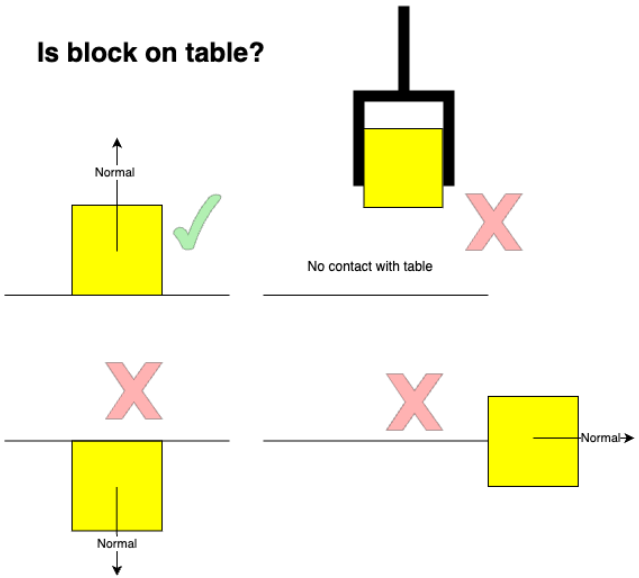
An **RGB-D** camera gives a color image and, for every pixel, the **depth** of the surface it shows.

How many blocks do you see?



Which image tells you that the stack is two blocks high?

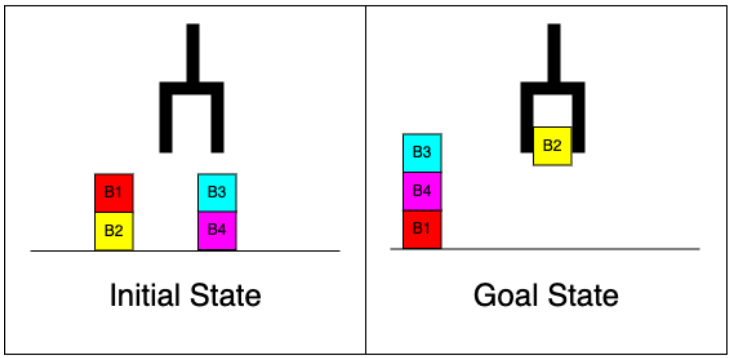
Checking the estimator: physics as ground truth

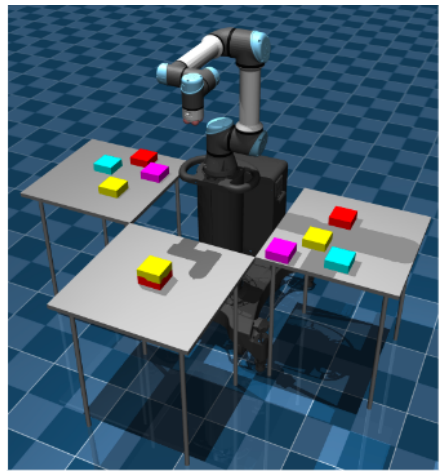


To *check* the estimator, we may cheat:

- MuJoCo computes the contacts between bodies
- A block is on whatever pushes it *up*
- Contacts exist only once the physics runs: `look()` moves the arm away, lets the scene settle, and observes

Read the plan as if you were the robot





`unstack(b2, b1)` how high above `b2`? from which side?
`put-down(b2, t1)` *where* on `t1`, a table behind a wall?
`pick-up(b3, t2)`
`stack(b3, b1)` what if `b1` moved since the plan was made?

The plan leaves this out on purpose: it keeps the planning problem small. We fill it in.

Challenges

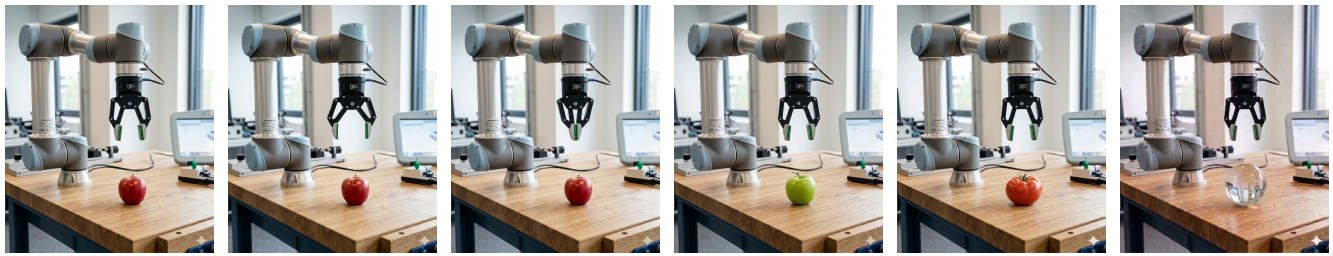
Is this the same observation?

Is this the same observation?

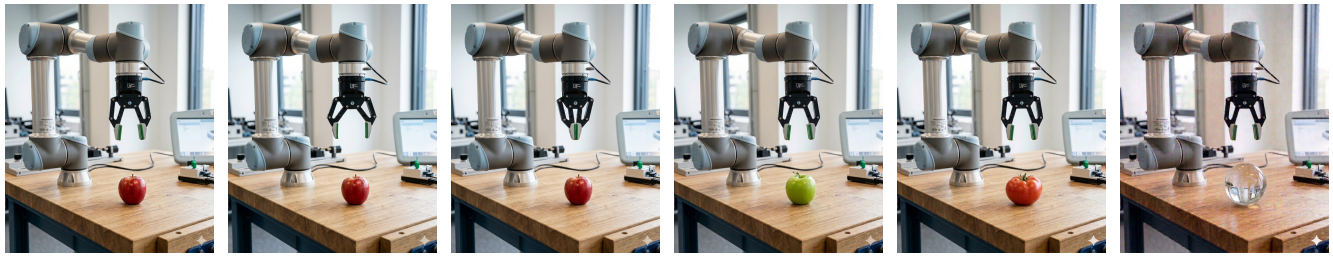
Is this the same observation?

Is this the same observation?

Is this the same observation?



BC assumes a consistent mapping from observations to actions - this breaks under ambiguity and distribution shift.

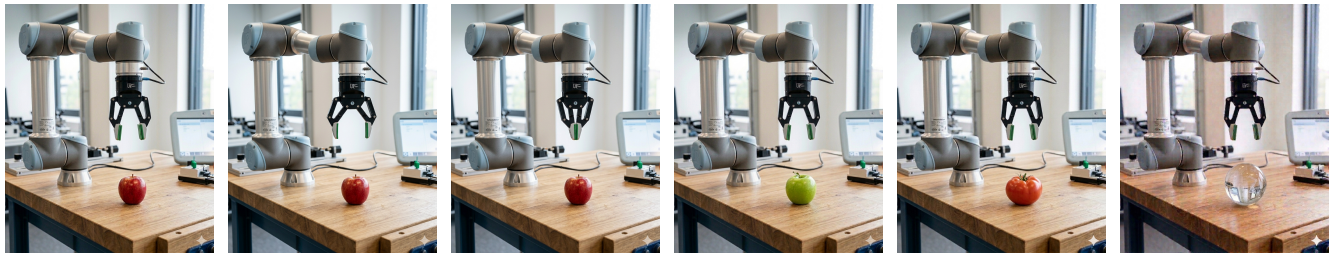


State aliasing \Perceptual aliasing (POMDP view): different underlying states produce visually similar observations, but require different actions:

$$o \approx o' \quad \text{but} \quad a^\star \neq a^{\star\prime}$$

In behavior cloning, the policy is learned as a direct mapping from observations to actions, i.e., $\pi(a \mid o)$.

In this case, conflicting supervision is recevied and classical methods tend to learn the **average action**: $\mathbb{E}[a \mid o]$ which can be suboptimal or even invalid.



The opposite can also occur: The same underlying state may appear visually different, and therefore be treated as out-of-distribution.

Covariate shift \Observation variability: The same state appears differently across contexts:

$$s = s' \quad \text{but} \quad o \neq o'$$

→ *observations fall outside training distribution*

Sim-to-Real Gap

Policies trained in simulation often fail on the real robot.

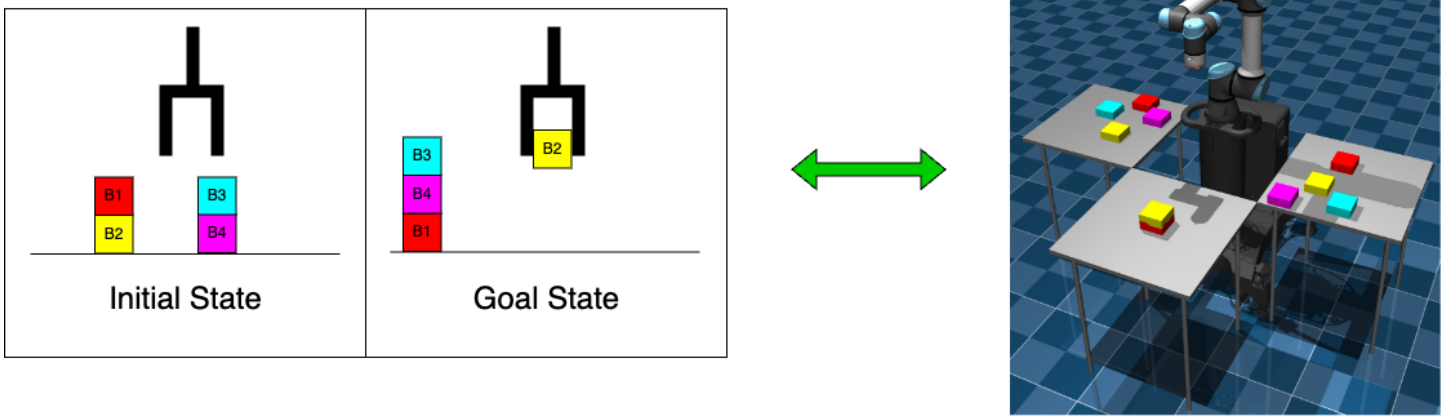
Why the gap exists

- **Dynamics** – friction, mass, contacts, motor delays modeled imperfectly
- **Perception** – rendered images differ from real cameras (lighting, textures, noise)
- **Unmodeled effects** – cable drag, wear, backlash, latency
- Policies exploit simulator *quirks* that don't exist in reality

Bridging the gap

- **Domain randomization** – vary physics and visuals so reality is "just another sample"
- **System identification** – calibrate the simulator to real measurements
- **Domain adaptation** – align sim and real features or images
- **Fine-tuning** – a little real-world data or online adaptation
- **Better simulators** –

Read the plan as if you were the robot




```
unstack(b2, b1)
put-down(b2, t1)
pick-up(b3, t2)
stack(b3, b1)
```

how high above `b2`? from which side?

where on `t1`, a table behind a wall?

what if `b1` moved since the plan was made?

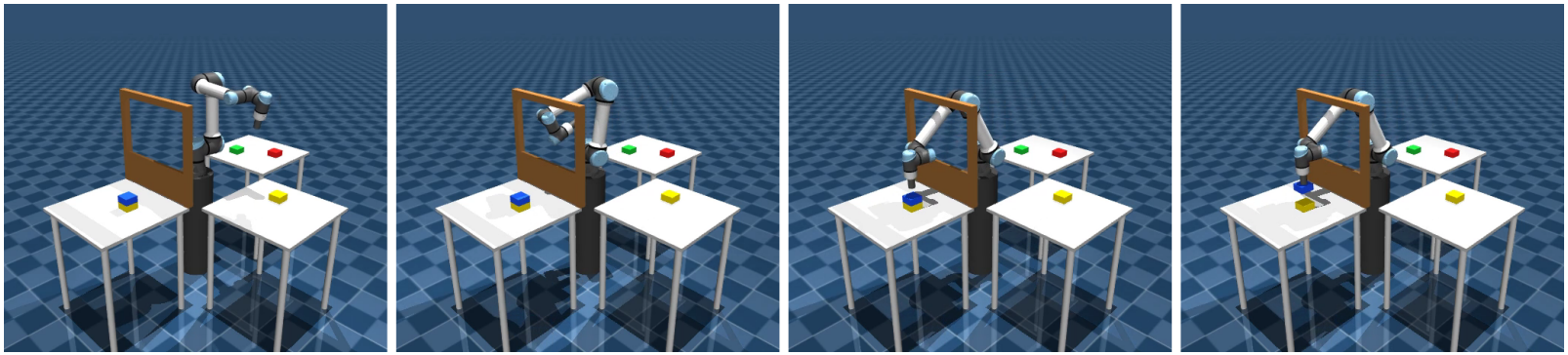
The plan leaves this out on purpose: it keeps the planning problem small. We fill it in.

Two worlds, two mappings

The task planner speaks in objects, facts and actions. The motion planner speaks in configurations.

- **State estimator**: world $\rightarrow$ facts. Before planning, after every action, at the end.
  (Today: the true state, read from the simulator's contacts)
- **Skills**: action $\rightarrow$ motions and gripper commands
- And the names: the planner's `b2` must be the simulator's `b2`

Skills: from actions to motions



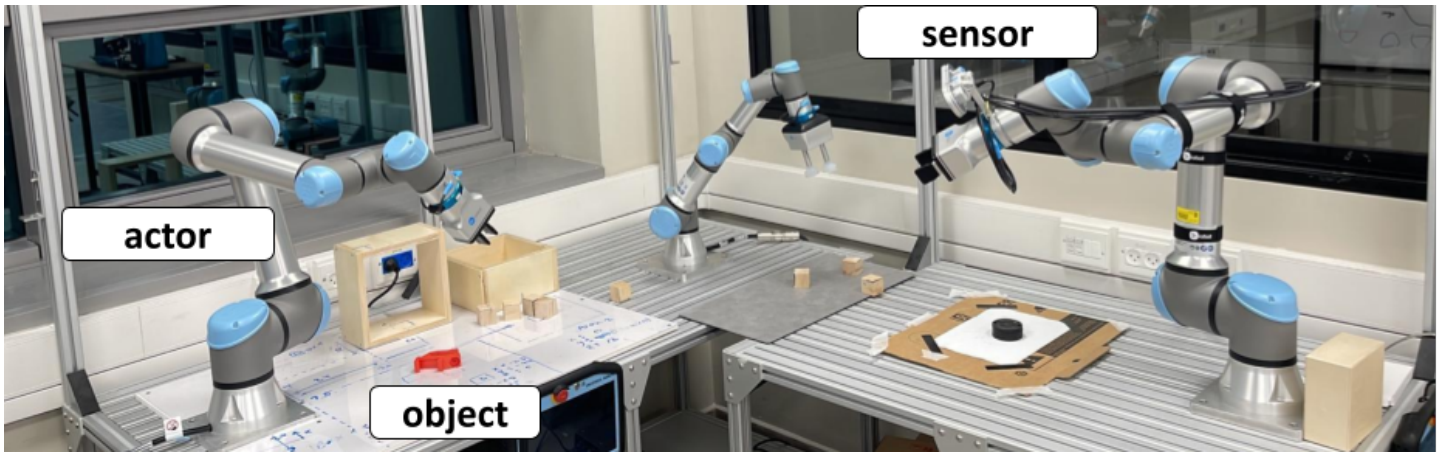
`pick-up`: **approach** (planned, through the window), **descend** (straight down), **grasp**, **lift**

Two changes from Lab 2.1, needed for long plans:

- Choose the grasp configuration first, then the approach configuration in the *same* IK family
- A skill reports failure and stops: no descent after a failed approach

CLAIR lab Examples

From Beliefs to Decisions



Value of Assistance for Grasping

Mohammad Masarwy, Yuval Goshen, David Dovrat, Sarah Keren

`https://arxiv.org/abs/2310.14402`

Belief Update

When receiving an observation o, the agent updates its current belief β using its **belief update function** $\tau : \mathcal{B} \times \Omega \times \mathcal{S} \mapsto \mathcal{B}$ which maps belief $\beta \in \mathcal{B}$ to the new belief.

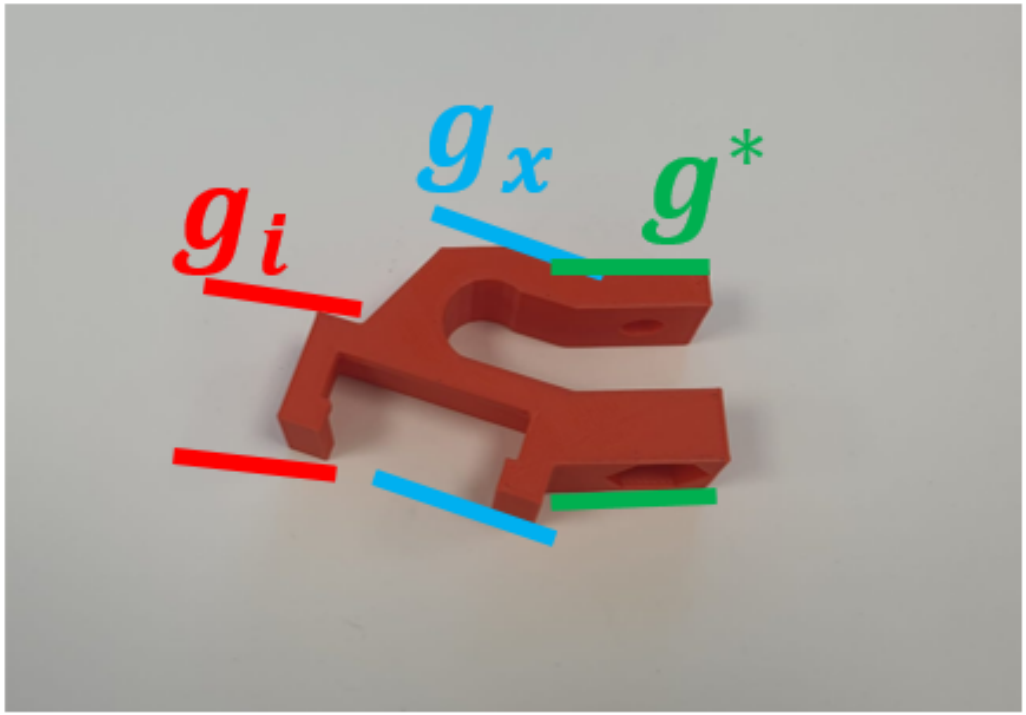
Commonly, a **Bayesian filter** is used:

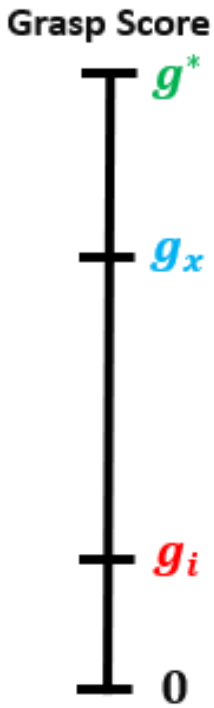
$$\beta^{o,a} = \frac{\mathcal{P}\left(o|s,a\right)\beta\left(s\right)}{\int_{s' \in \mathcal{S}} \mathcal{P}\left(o|s',a\right)\beta\left(s'\right)ds'} \tag{1}$$

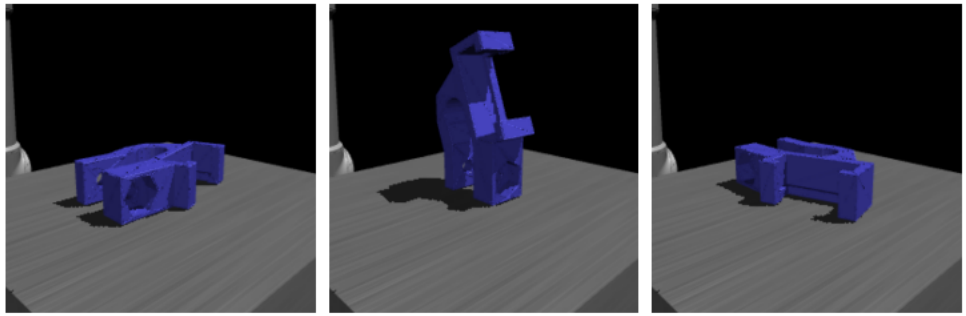
where $\mathcal{P}$ is the sensor function and $\beta\left(s\right)$ is the estimated probability that s is the actual world state
Discrete version:

$$\beta^{o,a} = \frac{\mathcal{P}\left(o|s,a\right)\beta\left(s\right)}{\Sigma_{p' \in \mathcal{P}_o} \mathcal{P}\left(o|s',a\right)\beta\left(s'\right)} \tag{2}$$

From Beliefs to Decisions

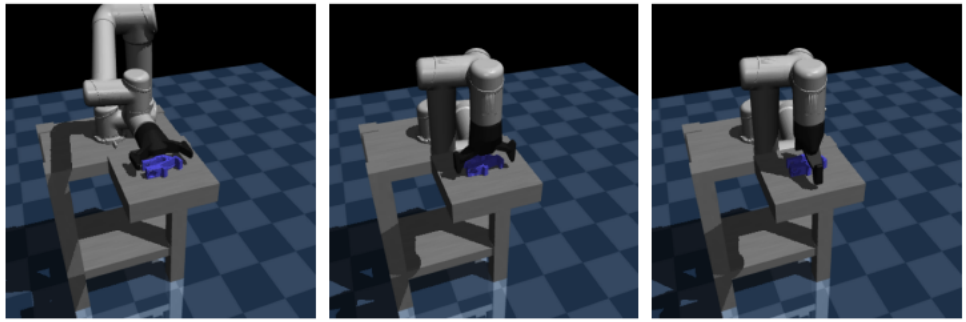






(a) (b) (c)

Fig. 2: Example stable poses.



(a) (b) (c)

Fig. 3: Example grasp configurations from which the actor can attempt to grasp the object - each configuration is associated with a score, i.e., probability of success.

From Beliefs to Decisions

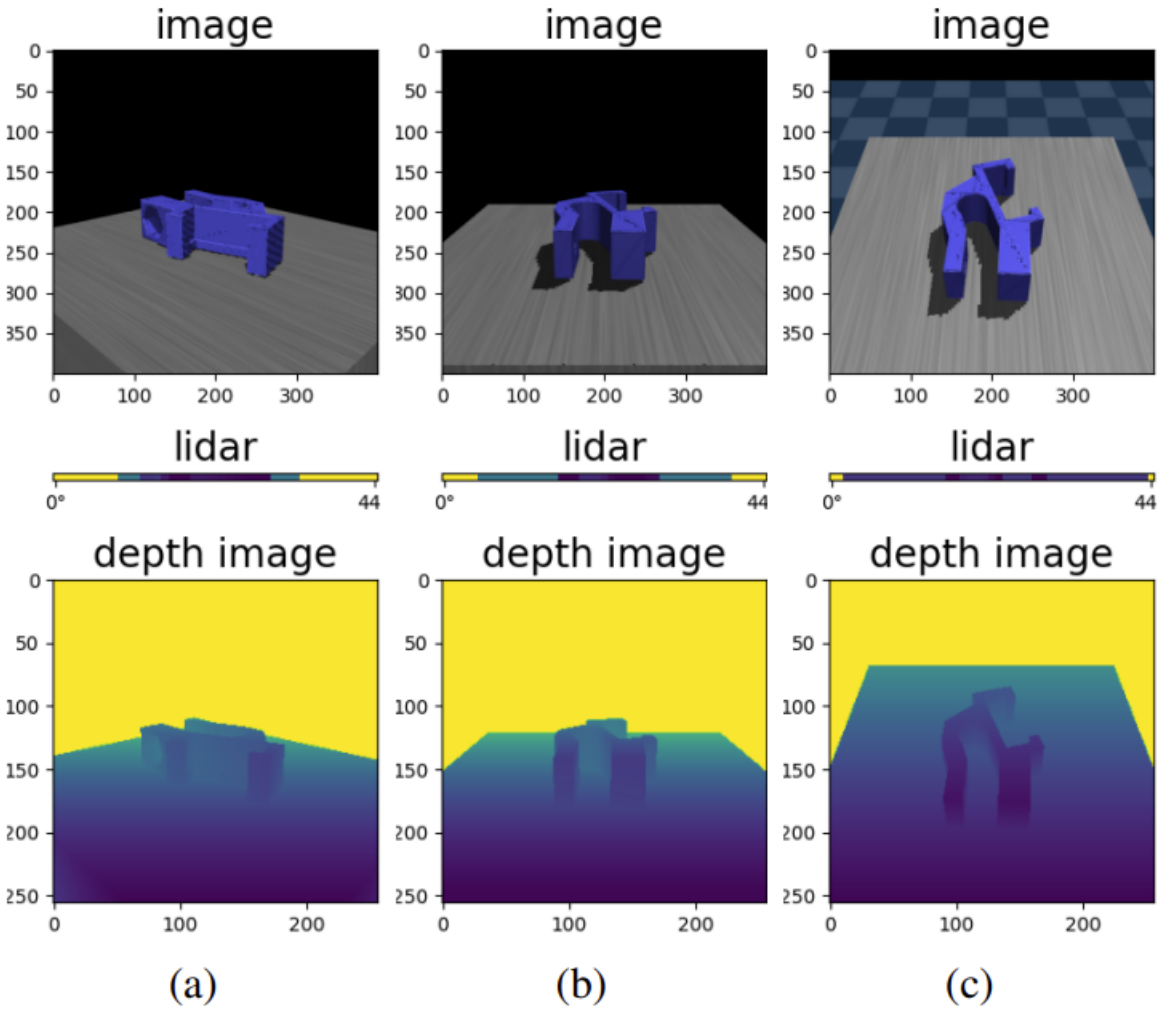


Fig. 4: Example sensor configurations and corresponding observations for a given stable pose. Each column represents the RGB image [top] lidar reading [middle] and depth image

Definition (Sensor Function)

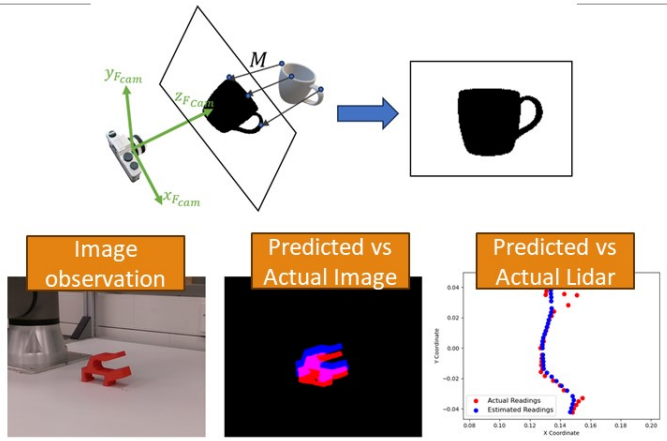
Given object pose $p \in \mathcal{P}_o$ and sensor configuration $q \in \mathcal{Q}$, sensor function $\mathcal{O} : \mathcal{P}_o \times \mathcal{Q} \mapsto \Omega$ is a random function, such that if $o = \mathcal{O}(p, q)$ then $P(o|p, q)$ provides the conditional probability of obtaining the observation o when the sensor configuration is q and the object pose is p.

Typically, the actual distribution is not known, and we use a *predicted sensor function* $\tilde{\alpha}$ and a *predicted observation probability* $\hat{P}$, which may be incorrect or inaccurate.

From Beliefs to Decisions

Challenge: the probability of an observation is near 0.

Predicted Observation Model - Example



From Beliefs to Decisions

We use a similarity score, $\omega : \Omega \times \Omega \mapsto [0, 1]$ to compare the predicted and received observations:

$$\hat{P}(o|p, q) = \frac{\omega(\tilde{\alpha}(p, q), o)}{\int_{p' \in \mathcal{P}_o} \omega(\tilde{\alpha}(p', q), o) dp'} \tag{3}$$

The literature is rich of various definitions for ω, which may vary between applications and sensor types.

For **belief update** we use a Bayesian filter such that for any observation $o \in \Omega$ taken from sensor configuration $q \in \mathcal{Q}$, the updated pose belief $\beta^{o,q}(p)$ for pose $p \in \mathcal{P}_o$ is given as

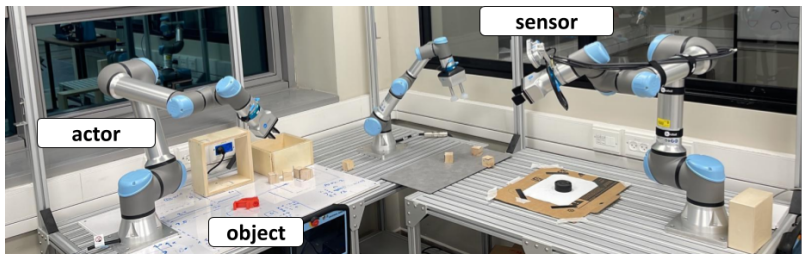
$$\beta^{o,q}(p) = \frac{\hat{P}(o|p,q)\,\beta(p)}{\int_{p' \in \mathcal{P}_o} \hat{P}(o|p',q)\,\beta(p')\,dp'} \tag{4}$$

where $\beta(p)$ is the estimated probability that p is the object pose prior to considering the new observation o.

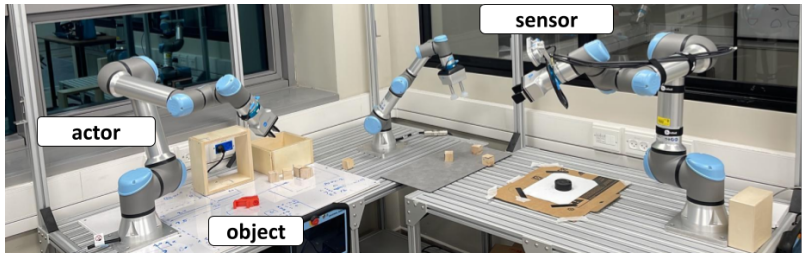
Value of Assistance (VOA) for Grasping

Given the actor's belief $\beta_a \in \mathcal{B}$, the belief of the helping agent $\beta_h \in \mathcal{B}$, the predicated observation probability $\hat{P}$, sensor configuration $q \in \mathcal{Q}$, and the actor's belief update function τ_a,

$$U_\alpha^{VOA}(\beta_h, \beta_{ac}) \stackrel{\text{def}}{=} \tag{5}$$
$$\mathbb{E}_{p\sim\beta_h}\left[\mathbb{E}_{o\sim\hat{P}(o|p,q)}\left[\gamma(q_g^*(\beta_a^{o,q}), p)\right] - \gamma(q_g^*(\beta_{ac}), p))\right].$$



From Beliefs to Decisions



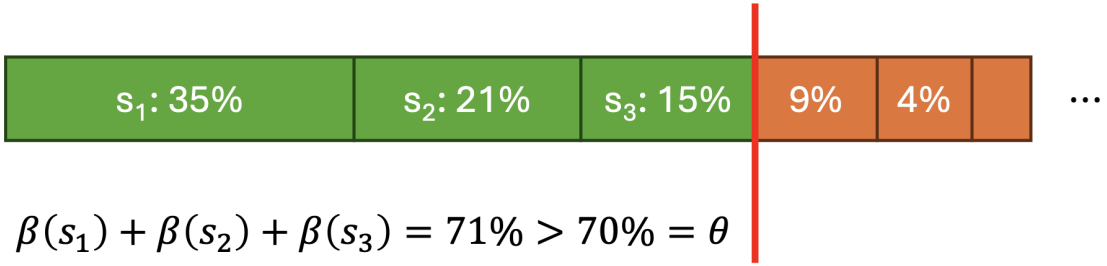
(a) P1 (b) P2 (c) P3 (d) P4 (e) P5 (f) P6

Is this a good observation?

Minimal Belief Sets

Don't use perception best-guess. Plan for multiple states. How many do we need?

$b = \{s_1, s_2, s_3\}$



$\beta(s_1) + \beta(s_2) + \beta(s_3) = 71\% > 70\% = \theta$

Probability-Ranked Valid State Search

$$Z = \sum_{s \vDash \mathcal{C}} \tilde{\beta}(s) = 1 - (\tilde{\beta}(110) + \tilde{\beta}(111)) = 0.72$$

Input

$\beta^F = [0.4, 0.7, 0.9]$
$\theta = 0.8$
$\mathcal{C} =$ first two features are mutually exclusive

\# - order of extraction from Q

1 011 $\tilde{\beta} = 0.6 \cdot 0.7 \cdot 0.9 = 0.378$, $\beta^{\mathcal{C}} = \frac{\tilde{\beta}}{Z} = 0.525$

2 111 $\tilde{\beta} = 0.4 \cdot 0.7 \cdot 0.9 = 0.252$, $\beta^{\mathcal{C}} = 0$ (Violates constraints)

3 001 $\tilde{\beta} = 0.162$, $\beta^{\mathcal{C}} = \frac{\tilde{\beta}}{Z} = 0.225$

010 $\tilde{\beta} = 0.042$, $\beta^{\mathcal{C}} = \frac{\tilde{\beta}}{Z} \approx 0.058$

drop visited

011

4 101

110

101

011

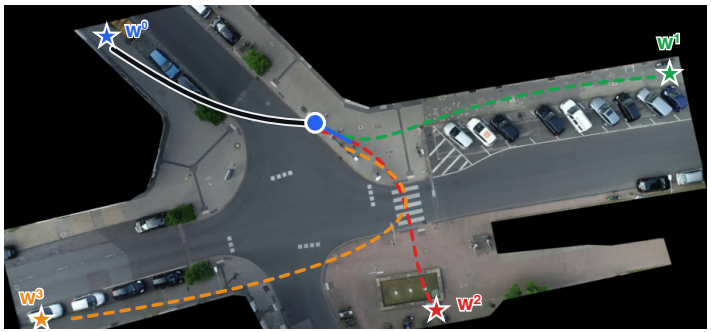
000

Goal Recognition under Uncertainty

Watching a robot move: *where is it going?*

Belief over goals:

$$P(g \mid o_{1:t}) \;\propto\; p(o_{1:t} \mid g)\, P(g)$$



- Same Bayesian update as the filter, but the state is the goal
- Likelihood from a learned diffusion prior over trajectories, with no hand-written dynamics model
- Keeps multimodal, calibrated beliefs on ambiguous prefixes